

Used Car Price Predictor

CSCI-335.01-02 | GROUP 3

Kai Fan • Dawn Grant • Quang Huynh

Rochester Institute of Technology
20 April 2026

The Problem

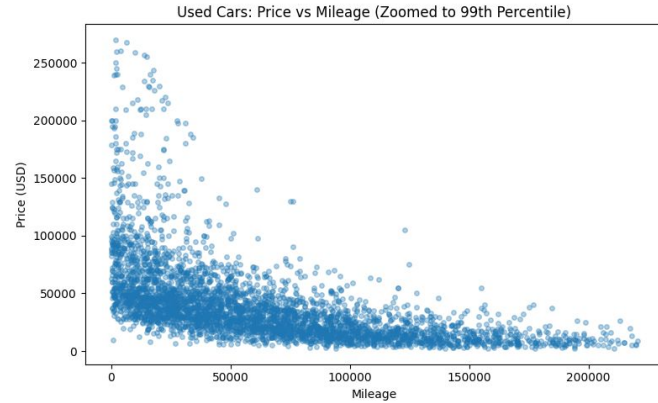
Predicting used car prices is difficult due to variability in features such as mileage, brand, condition, and market demand

Traditional pricing methods are inconsistent and often rely on subjective judgment

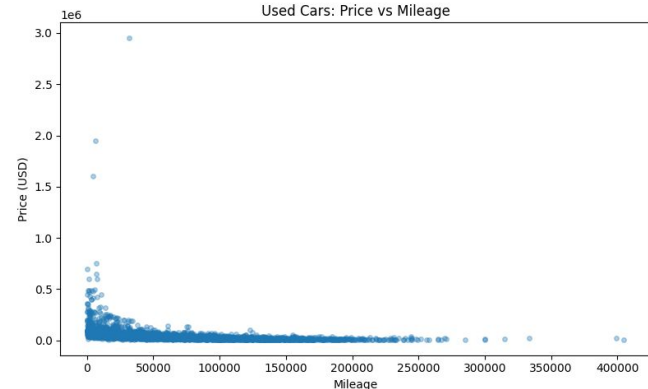
Accurate predictions can help buyers, sellers, and businesses make informed decisions

Our goal is to build a machine learning model that predicts car prices with high accuracy

Price vs Mileage Distribution



Sample Listing Price Variance



Background / Motivation

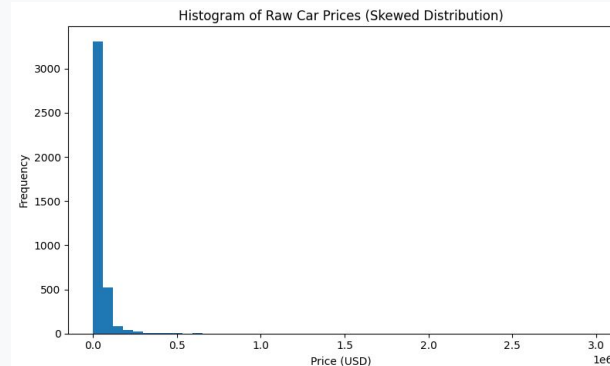
Used car prices are highly **non-linear and skewed**, making prediction challenging

Real-world datasets contain **noise, missing values, and outliers**

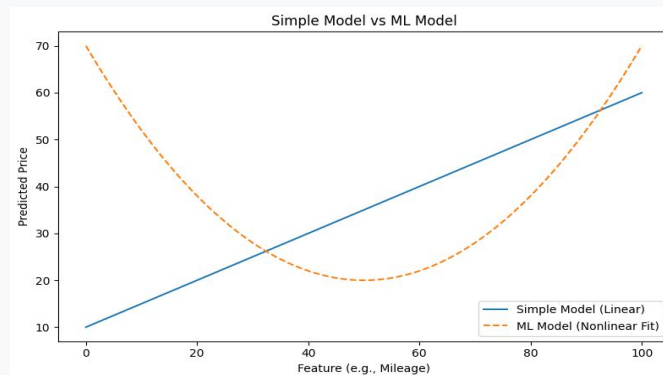
Machine learning allows us to model complex relationships **beyond simple regression**

We hypothesize that **preprocessing + model selection** will significantly improve performance

Histogram of Raw Price Distribution (Skewed)



Simple Model vs ML Model

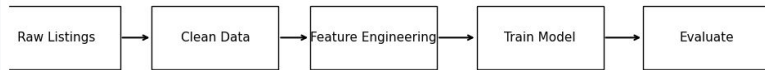


Hypothesis

- Removing outliers will reduce noise and improve model stability
- Log transformation of price will reduce skew and improve regression performance
- More advanced models (SVR, Random Forest) will outperform basic models
- Proper preprocessing is as important as model selection

Processing Pipeline

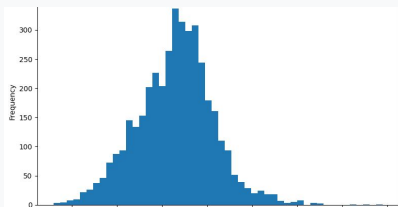
Used Car Price Prediction Pipeline



Raw Histogram (Before)



Log1p(price) (After)



Transformation Strategy

Raw Data → Outlier Removal → Log Transformation

Dataset

Overview

Used car listings with features:

- Price (target)
- Mileage
- Brand, model
- Fuel, transmission
- Engine info

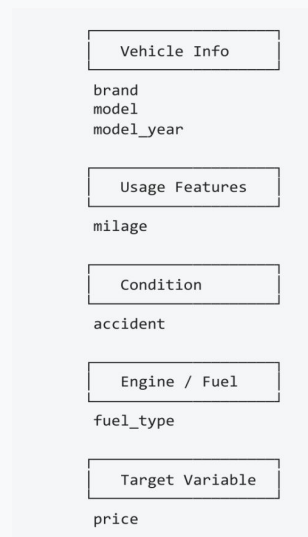
Scale: ~3,700+ entries (post-cleaning)

Nature: Real-world data with noise & inconsistencies

Table Preview (First 5 Rows)

brand	model	model_year	mileage	price	fuel_type	accident
Toyota	Camry	2018	45,000 mi	\$18,500	Gasoline	None reported
Honda	Civic	2020	30,200 mi	\$21,000	Gasoline	None reported
BMW	3 Series	2017	60,100 mi	\$25,300	Gasoline	Minor damage
Ford	F-150	2019	52,000 mi	\$27,800	Gasoline	None reported
Tesla	Model 3	2021	15,000 mi	\$38,900	Electric	None reported

Feature List Diagram



Data Preprocessing

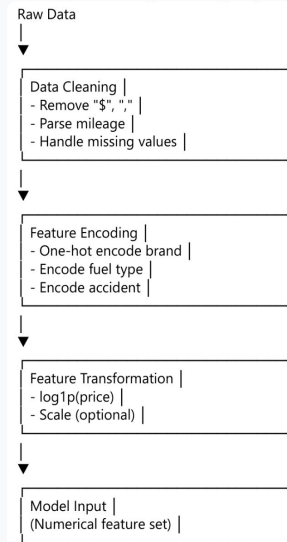
Preprocessing Steps

- Cleaned price column by removing symbols and converting to numeric
- Converted mileage from strings to numeric values
- Normalized categorical text features (lowercase, trimmed)
- Extracted new feature: engine size in liters
- Removed duplicate entries and rows with missing target values

Before vs After: Feature Transformation

Feature	Before (Raw Data)	After (Processed Data)
price	"\$18,500"	$18500 \rightarrow \log_{10}(\text{price})$
milage	"45,000 mi"	45000
accident	"None reported"	0
accident	"At least 1 accident"	1
fuel_type	"Gasoline"	One-hot encoded
brand	"Toyota"	One-hot encoded
model_year	"2018"	2018 (numeric)

Feature Engineering Pipeline



Outlier Handling & Target Transformation

1. Outlier Handling (IQR)

- Applied IQR-based filtering to remove extreme price outliers
- Reduced unrealistic or noisy entries that distort model training

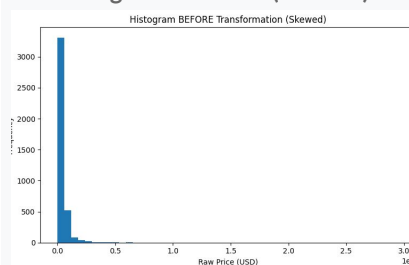
2. Log Transformation ($\log_{10}$)

Applied to price target variable:

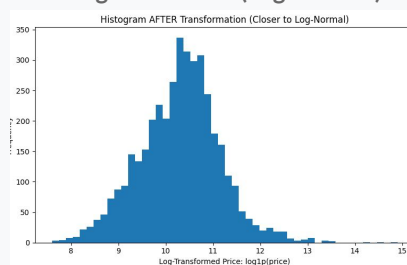
- Reduced skewness significantly
- Stabilized variance across the range
- Improved overall model performance

*Converted predictions back using `expm1` for evaluation

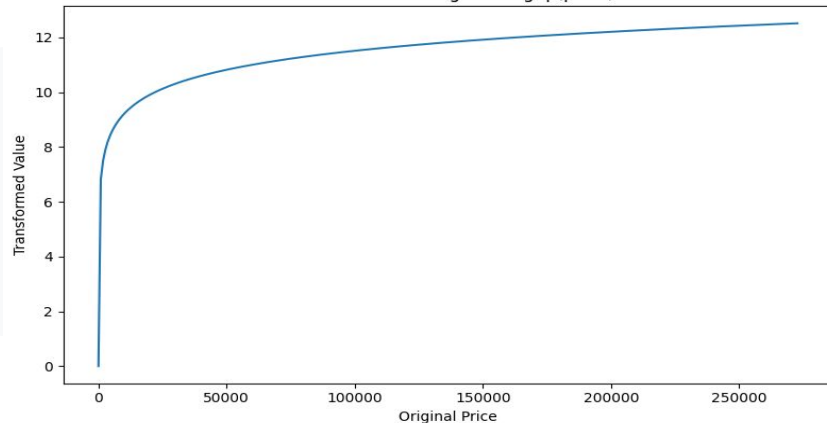
Histogram: BEFORE (Skewed)



Histogram: AFTER (Log-Normal)



Transformation Diagram: $\log_{10}(\text{price})$



Feature Processing

Numerical Features

- Imputed missing values using median
- Standardized using scaling

Categorical Features

- Imputed missing values using most frequent
- One-hot encoded for model compatibility

Used a **ColumnTransformer pipeline** to combine both feature sets seamlessly.

Pipeline Execution Flow

raw → num pipeline → scale → cat pipeline → encode → combined features

Models Used

Linear Regression (baseline)

K-Nearest Neighbors (distance-based model)

Decision Tree (non-linear model)

Random Forest (ensemble model)

Support Vector Regression (SVR)

Multi-layer Perceptron (Neural Network)

Models Trained

Linear Regression:

- Default parameter
- 80/ 20 split on training/ validation
- $R^2 = 0.7750$
- Average off by \$9975.54

KNN Regression:

- $K = 5, 7, 9$
- 80/ 20 split on training/ validation
- $K = 5$ $R^2 = 0.6503$
- $K = 7$ $R^2 = 0.6500$
- $K = 9$ $R^2 = 0.6410$
- $K = 5$ average off by \$12435.12
- $K = 7$ average off by \$12441.37
- $K = 9$ average off by \$12599.56

Models Trained

Support Vector Machine:

- Trained on both linear and rbf with $C = 0.1, 1, \text{ and } 10$
- 80/ 20 split on training/ validation
- Best performing: linear kernel, $C = 1$
- $R^2 = 0.8336$
- Average off by \$8577.33

Multilayer Perceptron:

- Hidden layer (32, 16), activation = relu
- 80/ 20 split on training/ validation
- $R^2 = 0.5324$
- Average off by \$14379.37

Ensemble Models

Random Forest:

- Trained on 50 and 100 n-estimators
- Trained on none, 5, and 10 max depth
- 80/ 20 split on training/ evaluating
- Best performing config: 50 n-estimator, none max depth
- $R^2 = 0.6521$
- Average off by \$12403.69

Decision Tree:

- Trained on none, 3, 5, and 7 max depth
- 80/ 20 split on training/ evaluation
- Best performing depth: 5
- $R^2 = 0.5289$
- Average off by \$14433.95

Evaluation Metrics

Metric Definitions

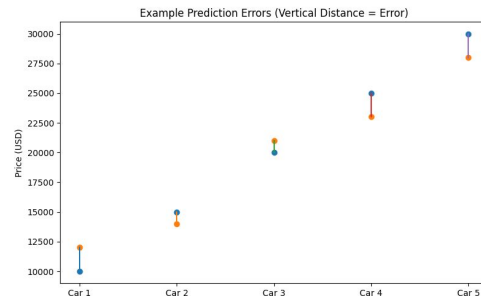
- **MSE**: Penalizes large errors heavily (squared difference)
- **RMSE**: Square root of MSE; interpretable in dollar units
- **MAE**: Average absolute difference between predictions and actual values
- **R²**: Proportion of variance explained by the model

"All metrics computed on the original dollar scale to maintain real-world interpretability."

METRIC DEFINITIONS TABLE

Metric	Definition
MSE	Penalizes large errors more heavily (squared differences)
RMSE	Square root of MSE, interpretable in original dollar units
MAE	Average absolute difference between predictions and actual values
R ²	Proportion of variance explained by the model

EXAMPLE ERROR VISUALIZATION



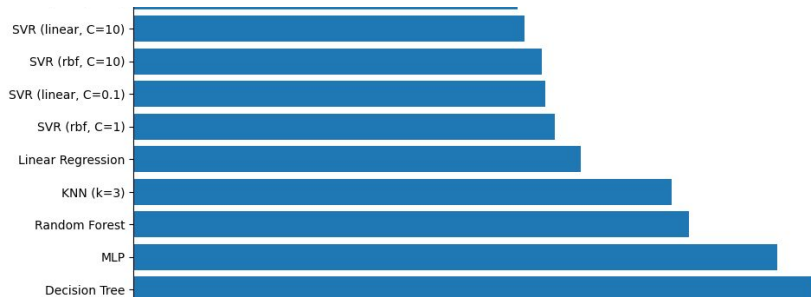
Model Comparison

Key Findings

- Models compared using **RMSE**, **MAE**, and **R²**
- **SVR** achieved best overall performance
- Linear models performed well after log transformation
- Tree-based models performed worse due to reduced variance

Interpretation: Linear relationships dominate after log transformation, benefiting SVR and linear regression.

R² SCORE COMPARISON (HIGHER IS BETTER)



RMSE COMPARISON



Best Model

SVR (Support Vector Machine)

Linear Kernel | C = 1

0.83

R² SCORE

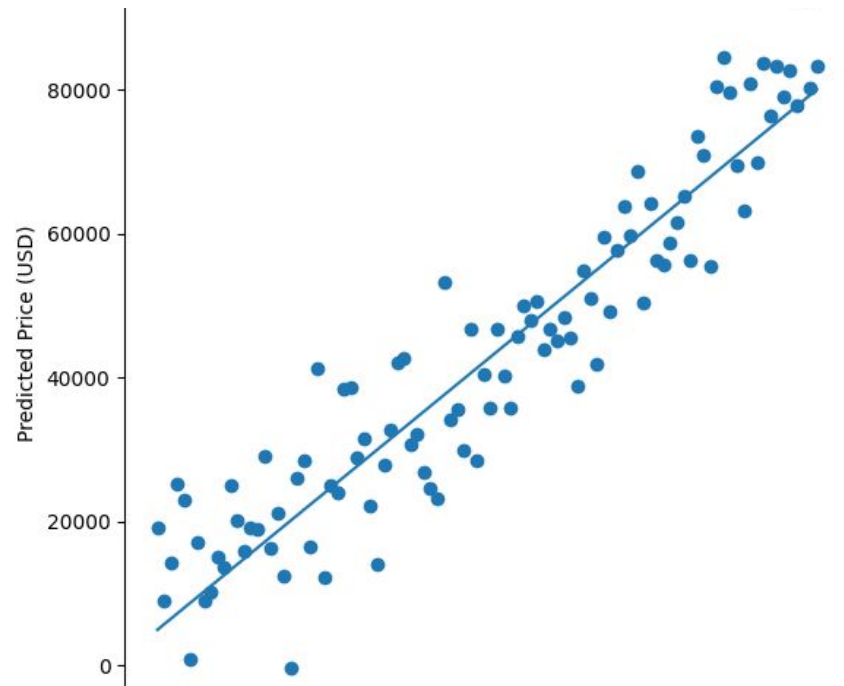
\$8,577

RMSE

\$5,540

MAE

Strong performance on noisy
real-world data



Actual vs Predicted Prices (\$)

Impact of Preprocessing

BEFORE PREPROCESSING

0.51_{R²}

+62%
IMPROVEMENT

AFTER PREPROCESSING

0.83_{R²}

KEY IMPROVEMENT DRIVERS



Outlier Removal



Log Transformation

Key Takeaways



Data Preprocessing

Preprocessing significantly impacts overall model performance and accuracy.



Log Transformation

Critical for handling skewed regression problems and stabilizing variance.



SVR Performance

Performed best due to its ability to capture smooth, complex feature relationships.



Real-World Complexity

Datasets require careful cleaning and advanced feature engineering for success.

Key findings from the machine learning pipeline and model evaluation phase.

Limitations



Feature Gaps

Dataset may not include all relevant features such as location or vehicle condition.



Data Dependency

Model performance is heavily dependent on the quality and cleanliness of source data. As well as data availability and amount of data.



Validation Method

A single train/validation split was used without rigorous cross-validation.



Overfitting Risks

Potential for model overfitting has not been fully explored or mitigated.

Key technical constraints and data limitations identified during the modeling phase.

Future Work



Robust Evaluation

Implement cross-validation techniques for a more robust and reliable model performance evaluation.



Feature Engineering

Expand the dataset by adding more relevant features like location, seller type, and vehicle condition.



Model Tuning

Perform advanced hyperparameter tuning using GridSearchCV to optimize model accuracy.



Productization

Deploy the final trained model as a user-accessible web application or a scalable API.

Strategic roadmap for enhancing model reliability and real-world deployment.

Acknowledgments



Team Members

Quang Huynh, **Kai Fan**, and **Dawn Grant** collaborated on all project aspects including preprocessing, modeling, evaluation, and presentation preparation.



Course Instructor

Suresh Pokharel provided guidance, feedback, and foundational knowledge in machine learning concepts used throughout this project.



Libraries Used

Relied on Python libraries: **scikit-learn** for modeling, **pandas** for data manipulation, and **numpy** for numerical computations.



Dataset Source

Used car price dataset from an online repository, featuring real-world data such as price, mileage, and vehicle specifications.



Questions